

Bloque 02 - Python desde cero: reglas verificables para eventos y pedidos

Propósito y resultado de salida

Python es un lenguaje para expresar instrucciones que un ordenador puede repetir. En análisis no se aprende para "programar por programar": se usa para convertir una regla de negocio -por ejemplo, «un pedido confirmado debe tener importe positivo»- en un proceso visible, repetible y comprobable.

Al terminar, Leo podrá leer, escribir y depurar un programa pequeño que reciba eventos o pedidos, valide casos anómalos, calcule resultados y explique qué datos descartó y por qué. Aún no trabajará con miles de filas ni con Pandas: primero dominará las piezas que luego Pandas automatiza.

Caso continuo: Lumen

Lumen es una app ficticia de comercio. Cada vez que una persona inicia o completa una compra se registra un **evento**: un pequeño diccionario con información como tipo, usuario e importe. Durante el bloque construiremos un auditor sencillo de pedidos: clasifica su estado, suma solo pedidos válidos y deja constancia de los errores. El caso permite ver que sintaxis, calidad de datos y decisión de negocio están unidas.



El mismo esquema aparecerá después en un notebook y, con estructuras más potentes, en Pandas. No se debe «arreglar» silenciosamente un evento: que sea inválido es información útil para producto e ingeniería.

Prerrequisitos y forma de estudio

Solo se presupone haber visto qué es un dato y una tabla en los bloques 00-01. Puede usarse [Google Colab](#) desde un navegador: crea un notebook, pega una celda, ejecútala y observa la salida. Un **notebook** mezcla texto, código y resultados; un **script** es un archivo .py que se ejecuta completo. Ambos usan Python; se elige notebook para explorar y script para repetir un proceso estable.

En cada lección: copia el ejemplo, predice su salida, ejecútalo, cambia un valor y explica qué regla se ha modificado. No avances si un mensaje de error aún parece misterioso: la lección 05 enseña a convertirlo en una pista.

Lecciones

1. [Entorno, valores, variables y expresiones](#)
2. [Colecciones: pedidos, copias y estructura anidada](#)
3. [Condiciones, operadores lógicos y bucles seguros](#)
4. [Funciones, contratos, módulos y alcance](#)
5. [Errores, pruebas y depuración basada en evidencia](#)
6. [Laboratorio: auditoría reproducible de pedidos](#)

Práctica verificable

1. Lee el [laboratorio ejecutable de Lumen](#). Puedes pegarlo por secciones en Colab o ejecutarlo con `python notebooks/practicas/02-auditoria-pedidos.py`.
2. Resuelve [la auditoría de pedidos](#) antes de mirar la [solución razonada](#).
3. El notebook histórico de gastos queda como práctica adicional, pero el caso de Lumen es la evaluación recomendada porque incluye entradas inválidas, límites y salidas esperadas.

Criterio de dominio

No basta con que el código «no falle». Debes poder señalar la entrada, la salida, la regla y al menos un caso límite para cada función. Si no sabes qué haría tu programa con None, un importe como texto, una lista vacía o un pedido exactamente en el umbral, todavía no está listo para automatizar decisiones.

01 - Entorno, valores, variables y expresiones

Resultado observable y prerequisites

Al finalizar podrás ejecutar una celda, predecir el resultado de una expresión y guardar cada resultado con un nombre que explique qué representa. No se requiere experiencia programando.

Del dato a una regla ejecutable

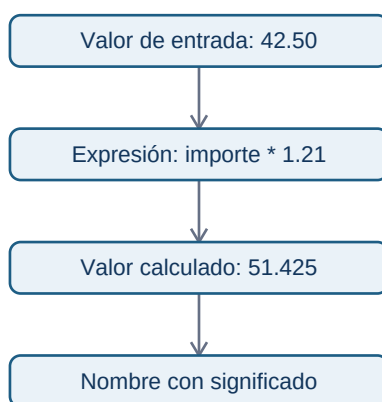
Imagina que Lumen recibe un pedido de 42,50 euros. Antes de hablar de «variables», mira una operación mínima:

```
42.50 * 1.21
```

Python evalúa esa **expresión** y devuelve 51.425. Una expresión combina valores y operadores para producir otro valor. Para análisis, conviene dar un significado a cada parte:

```
importe_sin_iva = 42.50
TIPO_IVA = 0.21
importe_con_iva = importe_sin_iva * (1 + TIPO_IVA)
print(importe_con_iva)
```

Una **variable** es un nombre que referencia un valor; = es una asignación, no una igualdad matemática. La constante en mayúsculas comunica una convención humana: no debería cambiar durante el cálculo. Python no impide modificarla, por lo que la revisión sigue siendo necesaria.



El diagrama recuerda que un nombre no convierte un dato en correcto: solo hace visible qué creemos que representa.

Tipos y operadores

Los tipos básicos son entero (3, int), decimal (3.5, float), texto ("web", str), booleano (True / False, bool) y ausencia explícita (None). `type(valor)` permite inspeccionarlos. Los operadores aritméticos incluyen +, -, *, / y % (resto); los comparadores ==, !=, <, <=, > y >= producen booleanos.

```
canal = "web"
```

```
importe = 42.50
es_web = canal == "web"
es_importe_positivo = importe > 0
print(es_web, es_importe_positivo) # True True
```

"42.50" + "10" da "42.5010": un texto. No conviertas un dato a float por reflejo; primero confirma que la fuente define ese texto como número y qué moneda usa.

Notebook, script y estado de ejecución

En un notebook ejecutas celdas en cualquier orden. Si ejecutas una celda que usa `importe` antes de aquella que lo crea, aparecerá `NameError`. En un script, Python lee las líneas de arriba abajo cada vez. Para aprendizaje, un notebook facilita experimentar; para una auditoría repetible, un script evita depender del orden oculto de clics.

Micropráctica

1. Predice y ejecuta `7 // 2`, `7 / 2` y `7 % 2`.
2. Crea `importe = "42.50"`. Compara `type(importe)` con `type(42.50)` y explica por qué no deben sumarse directamente.
3. Guarda `importe_original` y `importe_con_descuento` en nombres distintos. ¿Qué valor podrías auditar si sobrescribieras el primero?

Error frecuente y resumen

Evita nombres como `x` o `dato` cuando `importe_bruto` explica el significado. Tampoco confundas `=` con `==`: el primero guarda; el segundo compara. En la próxima lección varios valores formarán un pedido y una colección de eventos.

02 - Colecciones: pedidos, copias y estructura anidada

Resultado observable y prerequisites

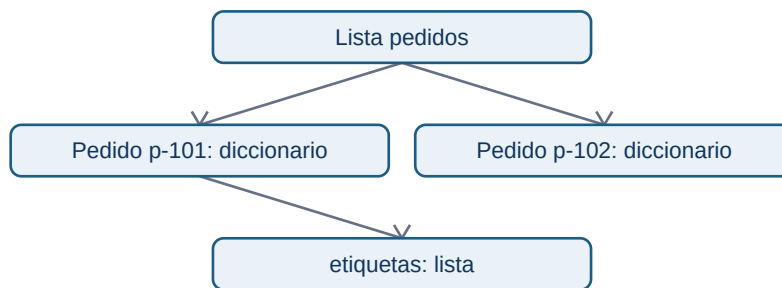
Al finalizar sabrás elegir lista, tupla, conjunto o diccionario para representar eventos de Lumen; accederás sin perder de vista índices, claves, copias y mutabilidad. Requiere valores y tipos de la lección 01.

Un pedido tiene partes; una colección conserva relaciones

Un diccionario asocia una **clave** con su valor. Una lista conserva una secuencia ordenada. Juntos pueden representar pedidos recibidos:

```
pedido = {"id": "p-101", "canal": "web", "importe": 42.50, "etiquetas": ["nuevo", "promo"]}
pedidos = [pedido, {"id": "p-102", "canal": "app", "importe": 18.00, "etiquetas": []}]
print(pedidos[0]["importe"]) # 42.5
```

`pedidos[0]` significa «primer elemento» porque Python empieza a contar desde cero.
`pedido["importe"]` usa una clave, no una posición. El anidamiento (lista dentro de dict) se parece a una respuesta JSON de una API; todavía no es una tabla, pero ya obliga a preguntar qué campos son obligatorios.



La relación no es una secuencia: un pedido tiene varios campos y una lista contiene varios pedidos.

Cuatro colecciones y su propósito

Un conjunto elimina duplicados: `set(["web", "web", "app"])` contiene dos canales. No lo uses si necesitas conservar cada evento: que dos pedidos compartan canal no los vuelve duplicados.

Slicing, mutabilidad y copias

El *slicing* toma una parte: `pedidos[0:2]` incluye posiciones 0 y 1; el extremo final no entra. Las listas y diccionarios son **mutables**: se pueden alterar después de crearse. Por eso esta aparente copia es peligrosa:

```
original = {"id": "p-101", "etiquetas": ["nuevo"]}
alias = original
alias["etiquetas"].append("revisar")
print(original["etiquetas"]) # también cambia
```

`alias` y `original` apuntan al mismo objeto. `original.copy()` copia solo el nivel exterior; para datos anidados usa `copy.deepcopy` cuando de verdad necesites independizar todos los niveles. Antes de copiar masivamente, pregunta si modificar el original es parte de la regla o un error.

```
from copy import deepcopy
pedido_limpio = deepcopy(original)
pedido_limpio["etiquetas"].append("auditable")
```

Microprácticas y límites

1. Crea tres pedidos y obtén los dos últimos con slicing.
2. Obtén los canales únicos con un conjunto. ¿Por qué el resultado no prueba cuántos pedidos hubo?
3. Prueba `pedido["descuento"]`: aparecerá `KeyError`. Después usa `pedido.get("descuento")` y explica la diferencia entre «no existe» y un descuento igual a cero.
4. Haz una copia profunda de un pedido con etiquetas, modifica la copia y verifica que el original no cambia.

No uses `get("importe", 0)` sin acordar su significado: sustituir ausencia por cero puede esconder un fallo de instrumentación. En la siguiente lección decidirás qué hacer con cada pedido mediante condiciones y bucles.

03 - Condiciones, operadores lógicos y bucles seguros

Resultado observable y prerequisites

Aplicarás una regla explícita a cada pedido, distinguiendo estados, y repetirás el proceso sin perder eventos. Requiere listas y diccionarios.

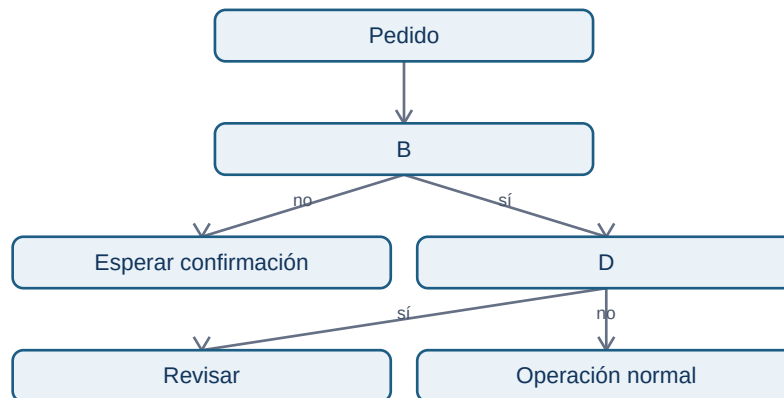
Una regla de negocio tiene bordes

Lumen considera revisable un pedido si su importe es al menos 100 EUR **y** está confirmado. Antes de codificar, decide qué pasa en los límites: 100 entra; 99,99 no; un importe desconocido no se aprueba automáticamente.

```
importe = 100
estado = "confirmado"

if importe >= 100 and estado == "confirmado":
    accion = "revisar"
elif estado != "confirmado":
    accion = "esperar confirmacion"
else:
    accion = "operacion normal"
```

`if` evalúa una condición, `elif` ofrece una alternativa y `else` cubre el resto. La sangría define qué instrucciones pertenecen a cada rama. Los operadores `and`, `or` y `not` combinan condiciones: `and` exige ambas verdaderas; `or` basta con una; `not` invierte. Usa paréntesis cuando mezcles condiciones para que la prioridad sea legible.



El orden de las preguntas importa: comparar un importe ausente antes de validar el estado o el tipo puede provocar un error o una decisión falsa.

Repetir: `for` cuando conoces la colección, `while` cuando esperas una condición

Un `for` visita cada elemento de una lista. Para sumar solo confirmados, se usa un acumulador que empieza en cero:

```
total_confirmado = 0
for pedido in pedidos:
    if pedido["estado"] == "confirmado":
        total_confirmado += pedido["importe"]
```

Un `while` repite mientras una condición sea verdadera. Es útil, por ejemplo, para reintentar una petición con límite; sin actualizar el contador puede no terminar nunca.

```
intentos = 0
MAX_INTENTOS = 3
while intentos < MAX_INTENTOS:
    intentos += 1
    print(f"Intento {intentos}")
```

No uses `while True` en un primer programa salvo que haya una salida clara (`break`) y una razón documentada. Para recorrer una lista de pedidos, `for` expresa mejor la intención.

break, continue y no modificar la lista recorrida

continue salta al siguiente elemento; break termina el bucle. Pueden ser correctos, pero un break tras el primer pedido inválido puede impedir auditar los demás. En análisis suele ser preferible guardar incidencias y continuar cuando el problema es local.

```
validos = []
incidencias = []
for pedido in pedidos:
    if not isinstance(pedido.get("importe"), (int, float)):
        incidencias.append(pedido.get("id", "sin_id"))
        continue
    validos.append(pedido)
```

No borres elementos de pedidos dentro del for: puedes saltarte posiciones. Construye validos y incidencias; así también conservas evidencia de la calidad de origen.

Microprácticas

1. Clasifica importes 99, 100 y 101. Explica por qué > no cumple la regla acordada.
2. Añade un pedido cancelado y otro con importe=None. Diseña qué lista debe recibir cada uno y justifica tu decisión.
3. Escribe un while de tres intentos y provoca deliberadamente el error de no incrementar el contador; no lo ejecutes sin un límite.
4. Recorre una lista y crea otra solo con canales web o app. ¿Cuándo usarías or?

Las condiciones hacen visibles decisiones; las funciones de la próxima lección impedirán repetir esas decisiones de manera inconsistente.

04 - Funciones, contratos, módulos y alcance

Resultado observable y prerequisites

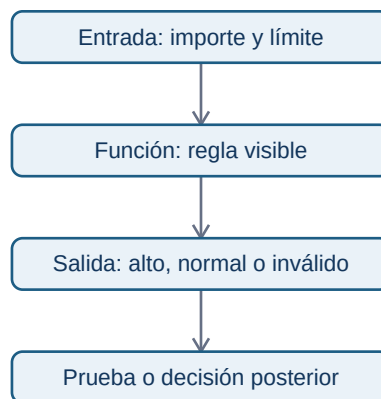
Crearás funciones pequeñas con entradas, salida y casos límite definidos; usarás argumentos opcionales, return, docstrings y comprobaciones. Requiere condiciones y bucles.

Una función es un contrato comprobable

Repetir `if importe >= 100` en varias celdas invita a que cada copia use un límite diferente. Una función nombra la regla y publica su contrato: qué acepta, qué devuelve y qué no hace.


```
def clasificar_importe(importe, limite=100):
    """Devuelve 'alto' si importe es numérico y alcanza limite; si no, devuelve 'invalido'."""
    if not isinstance(importe, (int, float)):
        return "invalido"
    if importe >= limite:
        return "alto"
    return "normal"
```

`importe` y `limite` son parámetros. `limite=100` es un argumento opcional: la persona que llama puede usar el valor por defecto o indicar otro. `return` entrega un resultado y termina la función; `print` solo lo muestra en pantalla. Por eso una función analítica debe normalmente devolver datos y dejar que otra parte decida cómo presentarlos.



Pruebas mínimas: normal, límite y dato inválido

Un ejemplo no demuestra que una regla funcione. Comprueba al menos un caso normal, el borde y una entrada inválida:

```
assert clasificar_importe(99) == "normal"
assert clasificar_importe(100) == "alto"
assert clasificar_importe("100") == "invalido"
assert clasificar_importe(150, limite=200) == "normal"
```

`assert` detiene la ejecución si la afirmación es falsa. No sustituye una suite profesional de tests, pero impide confiar en una salida bonita sin comprobar la regla. Un `AssertionError` es evidencia de que la expectativa y el código no coinciden.

Alcance y efectos secundarios

Los nombres creados dentro de una función son locales. Pasar datos como parámetros hace visible de qué depende la regla:

```
LIMITE_GLOBAL = 100
```

```
def es_revisable(importe, limite):  
    return importe >= limite
```

Evita que `es_revisable` lea `LIMITE_GLOBAL` sin recibirlo: cambiar una variable global en otra celda podría alterar resultados sin que la llamada lo muestre. Evita además modificar una lista recibida salvo que el contrato lo anuncie; es preferible devolver una nueva estructura o documentar el efecto.

Módulos, imports, script y notebook

Un **módulo** es un archivo Python que contiene funciones reutilizables. `import math` carga un módulo de la biblioteca estándar; `from math import ceil` importa un nombre concreto. No nombres tu archivo `math.py`, porque ocultaría el módulo oficial. Un script puede proteger su ejecución principal:

```
def main():  
    print(clasificar_importe(120))  
  
if __name__ == "__main__":  
    main()
```

Al ejecutar el archivo directamente se llama a `main`; al importarlo desde otro archivo se obtienen las funciones sin ejecutar el informe. Un notebook es útil para explorar; un módulo reduce copias cuando una regla ya está estable.

Microprácticas y resumen

1. Cambia el límite por defecto a 75 y prueba una llamada que lo sobrescriba.
2. Escribe `es_pedido_valido(pedido)` que devuelva `True` solo si contiene `id`, importe numérico positivo y estado confirmado.
3. Añade tres `assert` antes de confiar en ella.
4. Explica qué diferencia hay entre devolver una lista y hacer `print(lista)`.

Una función clara permite localizar fallos. La próxima lección enseña a distinguir los errores que Python señala y los resultados erróneos que Python no puede adivinar.

05 - Errores, pruebas y depuración basada en evidencia

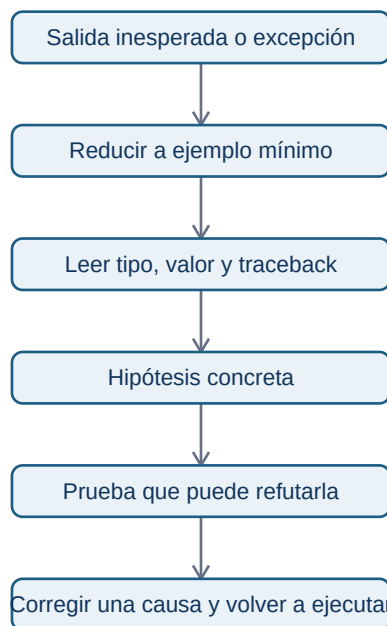
Resultado observable y prerequisites

Sabrás leer el final de un traceback, aislar un fallo, distinguir errores de sintaxis y de datos, y manejar solo excepciones esperadas. Requiere haber ejecutado funciones sencillas.

El traceback es un mapa, no una acusación

Un **traceback** es la ruta de llamadas que Python muestra cuando no puede continuar. La última línea nombra normalmente el tipo y la causa inmediata. Léela primero, luego vuelve a la línea de tu código indicada.

Un programa puede no lanzar excepciones y aun así estar equivocado: usar `>` en lugar de `>=` en el umbral de Lumen cambia la clasificación de 100 EUR sin que Python proteste. Por eso se combinan traceback y pruebas de casos límite.



La corrección se hace después de entender la causa; cambiar cinco líneas a la vez impide saber cuál resolvió el problema.

Validar cerca de la entrada

El laboratorio recibe eventos de una fuente simulada. Conviene transformar o rechazar un valor justo al llegar, no cuando el total ya es extraño:

```
def leer_importe(pedido):
    try:
        importe = float(pedido["importe"])
    except KeyError as error:
        raise ValueError("El pedido no contiene importe") from error
    except (TypeError, ValueError) as error:
        raise ValueError("El importe debe ser numérico") from error
    if importe <= 0:
        raise ValueError("El importe debe ser positivo")
```

```
return importe
```

Capturamos excepciones concretas porque son esperables en esta frontera. `except Exception:` `pass` sería mala práctica: silencia también problemas de programación y puede dejar un informe incompleto sin aviso. La excepción se traduce a un mensaje de negocio, pero se conserva la causa con `from error`.

assert y registro de incidencias

Usa `assert` para afirmar un comportamiento que el programador espera durante desarrollo. Para datos que vienen de fuera, una excepción controlada o una incidencia suele ser mejor que detener toda la auditoría:

```
incidencias = []
for pedido in pedidos:
    try:
        importe = leer_importe(pedido)
    except ValueError as error:
        incidencias.append({"id": pedido.get("id", "sin_id"), "motivo": str(error)})
        continue
```

Esto no «arregla» el origen. Permite calcular un total de pedidos válidos y comunicar cuántos se excluyeron. Un analista debe informar ambos números; de lo contrario, la cifra parece más precisa de lo que es.

Microprácticas

1. Provoca un `SyntaxError` eliminando dos puntos tras `if`; restaura el código y explica qué esperaba Python.
2. Crea una lista con dos pedidos e intenta acceder a `pedidos[2]`. ¿Por qué es `IndexError` y no `KeyError`?
3. Haz que `leer_importe({"importe": "--"})` lance `ValueError`; comprueba el mensaje.
4. Escribe un `assert` para el borde: un importe 0 no es válido.

Depurar consiste en producir evidencia sobre una hipótesis. En el laboratorio final reunirás datos válidos, incidencias, pruebas y una salida que otra persona pueda revisar.

06 - Laboratorio: auditoría reproducible de pedidos

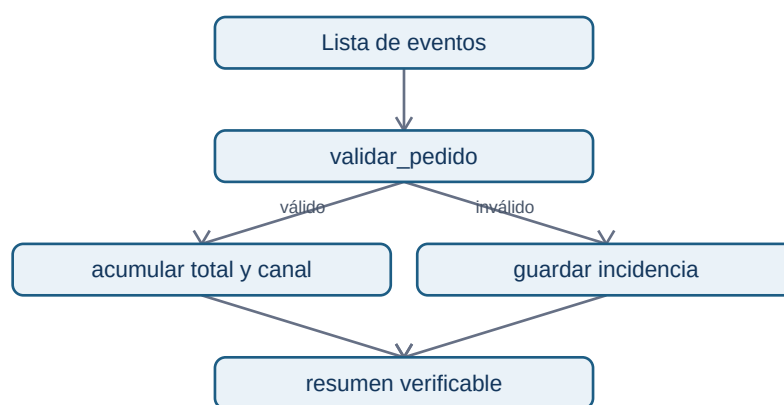
Resultado observable y prerequisites

Construirás y ejecutarás una auditoría pequeña de Lumen: entradas simuladas, reglas explícitas, incidencias, totales y pruebas. Requiere las cinco lecciones anteriores.

El problema operativo

Producto quiere conocer el importe confirmado del día. Ingeniería advierte que algunos eventos llegan con texto en lugar de número, importe cero o estado inesperado. Informar únicamente un total sería engañoso: también hay que comunicar qué quedó fuera y por qué.

El [laboratorio ejecutable](#) separa cuatro responsabilidades:



Separar funciones no es estética: permite probar la validación sin depender de la presentación del resumen.

Lectura guiada del laboratorio

Primero observa `PEDIDOS_DEMO`. Es una lista deliberadamente pequeña con pedidos correctos, un importe de texto convertible, un importe cero, una clave ausente y un pedido cancelado. Después ejecuta las pruebas `assert`: si una falla, no continúes al resumen; la regla ha cambiado o está mal implementada.

La función `validar_pedido` devuelve una copia normalizada del pedido válido o lanza `ValueError` con una explicación. `auditar_pedidos` no oculta la excepción: la convierte en una incidencia con `id` y `motivo`. La salida esperada de los datos demo es:

```
Pedidos válidos confirmados: 2
Importe confirmado: 160.50 EUR
Por canal: {'web': 120.5, 'app': 40.0}
Incidencias: 4
```

El pedido cancelado aparece como incidencia porque este informe tiene el contrato «solo confirmados». En otro informe podría contarse como estado separado; no hay una respuesta universal, sí debe haber una definición visible.

Estilo que protege el análisis

- Usa nombres que expresen unidad y significado: `importe_confirmado`, no `resultado`.
- Declara límites (`LIMITE_REVISION = 100`) en lugar de números mágicos repartidos.
- No mezcles lectura, validación, cálculo y `print` en un bucle enorme.
- Devuelve datos desde las funciones; imprime solo en el borde del programa.
- Conserva incidencias y recuentos. Un valor excluido sin rastro es una fuga de trazabilidad.

No basta con que una salida coincida una vez. Cambia un pedido demo para que valga 100, añade un canal nuevo y prueba una lista vacía. El programa debe responder de forma definida, no por accidente.

Ejercicio de cierre

Resuelve [Auditoría de pedidos de Lumen](#). Incluye un estado adicional y un límite de revisión; luego compara tu razonamiento con la [solución](#). Si estudias desde móvil, copia por partes el script en Colab o redacta primero el contrato de cada función, sus entradas y salidas esperadas.

Puente a NumPy y Pandas

Aquí recorres una lista pedido por pedido para comprender el control. NumPy y Pandas aplicarán operaciones parecidas a muchas filas, pero las preguntas no desaparecen: ¿qué es válido?, ¿qué se excluyó?, ¿qué significa cero?, ¿puedo reproducir el resultado? Lleva estas preguntas al bloque 04 y, especialmente, al bloque 05.